JAVASCRIPT ARRAYS

PROG1123

A COMPARISON TO WHAT YOU ALREADY KNOW

- When you see comma-separated lists in square brackets in programming, you classify it as list-like in nature
- Usually, lists can contain many types of things: numbers, strings, dates, objects etc
- Typically with list-like structures, we can access the length of the list and we can index separate elements in a list, using a numeric value
- NodeLists and Strings in JavaScript fit this loose and broad definition of a list

```
//example of how a string is list-like
//it has a length property
let word = "hello"
console.log(word.length)
```

```
5
```

```
//access the 3rd element in the string
console.log(word[2])
```

```
l
```

```
▶ NodeList(3) [input#chkExtraCheese, input#chkExtraSauce, input#chkAnchovies]
```

JS ARRAY DEFINITION

- Arrays, too, are lists in nature, but they have a distinct & broader set of properties and methods that allow us to manipulate and traverse through the items in the list
- Arrays in JavaScript can hold primitive and non-primitive data types
- The length of an array in JS is not fixed, we can dynamically add more items to arrays
- Here is how you declare an array in JS:

```
//declare an empty array:
let cars = [];

//declare an array of strings
let trucks = ["F150", "Durango", "Silverado"];
```

JS PROTOTYPES

- When a variable references a particular data type, it inherits what is called a prototype
- When something in JS inherits a prototype, it automatically "gets" all of the properties and methods that belong to the prototype

```
//this reference inherits a NodeList prototype
▼NodeList(3) [input#chkExtraCheese,
  ▶0: input#chkExtraCheese
  ▶1: input#chkExtraSauce
  ▶2: input#chkAnchovies
   length: 3
  ▶__proto__: NodeList
 trucks

//this reference inherits the Array Prototype
▼(3) ["F150", "Durango", "Silverado"]
   0: "F150"
   1: "Durango"
   2: "Silverado"
   length: 3
  ▶__proto__: Array(0)
```

LEARNING MORE ABOUT A JS PROTOTYPE

- If you are in the console, you can drill down through the prototype list to inspect the methods and properties that you can access and use in your problem-solving
- Or, you can do some online research on JS Array Prototypes via MDN

MDN web docs
moz://a

Array.prototype

```
1: "Durango"
2: "Silverado"
length: 3
▼ __proto__: Array(0)
  ▶ concat: ƒ concat()
  ▶ constructor: ƒ Array()
  ▶ copyWithin: ƒ copyWithin()
  ▶ entries: ƒ entries()
  ▶ every: ƒ every()
  ▶ fill: ƒ fill()
  ▶ filter: ƒ filter()
  ▶ find: ƒ find()
  ▶ findIndex: ƒ findIndex()
  ▶ flat: ƒ flat()
  ▶ flatMap: ƒ flatMap()
  ▶ forEach: ƒ forEach()
```

THE ANATOMY OF AN ARRAY

- The beginning of an array is called the **head** and the end of it is called the **tail**
- Each item in the array is called an **element**
- It's position in the array is called its **index**

```
let trucks = ["F150", "Durango", "Silverado"];

//access the head of the array
console.log(trucks[0]);

//access the tail of the array
console.log(trucks[trucks.length - 1]);
```

```
F150
Silverado
```

```
let trucks = ["F150", "Durango", "Silverado"];
//add new truck to end/tail
trucks.push("Ram 1500");

//add new truck element to the beginning
trucks.unshift("Colorado");

//add a truck to a specific position within the array.
//the first param indicates where and the second
//parameter says how many to delete with
//the inserting
trucks.splice(3,0,"Sierra");

console.log(trucks);
```

```
▶ (6) ["Colorado", "F150", "Durango", "Sierra", "Silverado", "Ram 1500"]
```

ADDING ELEMENTS TO AN ARRAY

push() adds a new element to the tail

unshift() moves an element to the head

splice() moves an element to any index position of your choosing

ALTERNATIVE USES OF PUSH(), SHIFT() AND SPLICE()

- In the previous slide, you saw how you could use push(), splice() or shift() to add new elements to an array.
- In this slide you are shown that you can add many new elements at one time!

```
let trucks = ["F150", "Durango", "Silverado"];
trucks.unshift("Colorado", "Denali");

//add a truck to a specific position within the array.
//the first param indicates where and the second
//parameter says how many to delete with
//the inserting
trucks.splice(3,0,"Sierra", "Canyon");

console.log(trucks);
```

```
VM2975:10
▶ (7) ["Colorado", "Denali", "F150", "Sierra", "Canyon", "Durango", "Silverado"]
```

REMOVING ELEMENTS FROM AN ARRAY

- You can use pop() to remove an element from the tail
- If you call the shift() method without passing it an element, it removes the first element
- You can use splice() and pass it a number of items to remove at a particular position
- There are other ways that we can remove items too, and we will cover that soon!

```
console.log(trucks);
```

```
▶ (7) ["Colorado", "Denali", "F150", "Sierra", "Canyon", "Durango", "Silverado"]
```

```
//pop() will also return the item that was removed
//so you can capture it in an assignment statement
let lastItem = trucks.pop();
console.log(lastItem);

//get rid of the first element
trucks.shift();
console.log(trucks);

//get rid of the 2nd and 3rd elements
//start at index 2 and remove 2 items
trucks.splice(1, 2);
console.log(trucks);
```

```
Silverado
▶ (5) ["Denali", "F150", "Sierra", "Canyon", "Durango"]
▶ (3) ["Denali", "Canyon", "Durango"]
```

CHANGING HOW ELEMENTS ARE ORDERED IN THE ARRAY

- Sorting an array is done in place. This means that in memory, the actual array, is sorted, not a copy of the array
- You can flip the elements so that the tail is now the head and visa versa, too!

```
let trucks = ["F150", "Durango", "Silverado"];
let prices = [59000, 90000, 75000];

//you can sort an array. Strings sort A-Z and
//number types sort 0-9
trucks.sort();
prices.sort();
console.log(trucks);
console.log(prices);
```

```
▶ (3) ["Durango", "F150", "Silverado"]
▶ (3) [59000, 75000, 90000]
```

```
let trucks = ["F150", "Durango", "Silverado"];
//you can also reverse the order of the array!
trucks.reverse();
console.log(trucks);
```

```
▶ (3) ["Silverado", "Durango", "F150"]
```

DETERMINING IF AND WHERE A VALUE IS IN AN ARRAY

- The indexOf() method searches for a value in a JS array.
- If a match is found, the first index where it is found is returned; otherwise, negative one (-1) is returned

```
let trucks = ["F150", "Durango", "Silverado"];
let foundPos = trucks.indexOf("Silverado");

if (foundPos != -1){
    console.log(`A match was found. It is element # ${foundPos + 1}`);
}
else{
    console.log("No match found");
}
```

```
A match was found. It is element # 3
```

ITERATING THROUGH AN ARRAY

- The for … of loop permits us to move through array elements one at a time from the head to the tail.

```
let trucks = ["F150", "Durango", "Silverado"];

for(t of trucks){
    console.log(t);
}
```

```
F150
Durango
Silverado
```

WAIT! THERE'S MORE!

- There are so many array methods in JS. We will turn to several other methods after we play with some of these basic ones!
- Now its time to practice these methods with some programming challenges